

קורס - נושאים מתקדמים בשפת C

קובץ בדיקות

ניר שרעבי 213063969

אסף חיון 214195331

ת.ה: 04.08.2025

Header and Source files

car.h

```
#ifndef CAR_H
#define CAR_H

#include "port.h"
#include "types.h"
#include <stdbool.h>

typedef struct Car {
    char nLicense[9];
    portType portType; //type of charging
    double totalPaid; //total amount paid for charging
    struct Port* pPort; // pointer to the port the car is connected to;
    int inqueue; // 1 if car is inqueue, 0 otherwise
}Car;

typedef struct tCar {
    Car* p2car; // pointer to car
    struct tCar* right; // right child
    struct tCar* left; // left child
}tCar;

/**
 * manager for car BST
 */
typedef struct {
    tCar* root; //root of the BST
    int totalCars; //total number of cars in BST
}CarBST;

//function prototypes
//-----
/**
 * Creates and initializes a Car
 * @param license Car license string
 * @param portType Charging type(FAST,MID,SLOW)
 * @return pointer to new Car
 */
Car* createCar(const char* license, int portType);

/**
 * Prints the car information
 */
void printCar(const Car* c);

/**
 * Returns 1 if car is not connected and not inqueue-safe to delete.
 */
int isCarAvailableForDeletion(const Car* c);

/**
 * Creates and initializes an empty CarBST
```

```

*/
CarBST* createCarBST();

/**
 * inserting a car into the BST by license
 * @return 1 if successful, 0 if duplicate or error
 */
int insertCar(CarBST* CarTree, Car* newCar);

/**
 * Searches for a ca by license.
 * @return Pointer to the car if found, NULL otherwise
 */
Car* searchCar(CarBST* CarTree, const char* license);

int countCarNodes(const tCar* root);
/**
 * counts the number of cars in th BST=
 */
int countCars(const CarBST* CarTree);

/**
 * Frees all memory used by the car BST
 */
void freeCarsTree(CarBST* CarTree);

/**
 * Removes a carNode from the car tree based on the license
 * @param node pointer to the root node of the car tree
 * @param license string of the car to be removed
 * @return Pointer to the updated root of the car tree
 *
 * The function searches for a node with matching license. if found,it removes
 * the node while keeping the bst as he should be, and free its memory.
 * if not matching node is found, the original tree remains unchanged.
 */
tCar* removeCarNode(tCar* node, const char* license);

/**
 * Calculates the total charging time in minutes for a car since it was plugged in.
 * @param car Pointer to the Car struct that has the charging port information.
 * @param now_time The current date and time.
 * @return The total number of minutes the car has been charging.
 *
 * The function converts both the car's plug-in time and the current time into minutes,
 * then returns their difference.
 */
int totalChargeMinutes(Car* car, Date now_time);

/**
 * Save Cars to dynamic array in order from the car tree
 * @param root Pointer to the root of the car tree
 * @param arr Pointer to a dynamically allocated array of the car pointers to store.
 * @param index Pointer to an integer that track the current index of the array
 *
 * The function performs an in-order recursively on the car tree,
 * and populates the array with pointers to car structs in sorted order.

```

```

*/
void saveCarsInOrder(const tCar* root, Car** arr, int* index);

/**
 * Compare function for sorting Car pointers by total amount paid.
 * @param a Pointer to a pointer to the first Car object
 * @param b Pointer to a pointer to the second Car object.
 * @return An integer indicating the comparison result:
 * -Returns a positive value if c2->totalPaid > c1->totalPaid
 * -Returns a negative value if c2->totalPaid < c1->totalPaid
 * -Returns 0 if both have equal totalPaid
 *
 * this function was written for use with function 'qsort',
 * comparing two Car* elements stored in an array.
 * It handles NULL pointers by returning 0.
*/
int compareByPaid(const void* a, const void* b);

```

```

#endif //CAR_H

```

car.c

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "car.h"
#include "helper_functions.h"

#define LICENSE_SIZE 9

//internal functions helpers
// Internal helpers for managing tCar tree nodes
static tCar* insertCarNode(tCar* root, Car* newCar); //Recursively inserts a car node
static Car* searchCarNode(tCar* root, const char* license); // search car by license
static void freeCarNodes(tCar* root); // freeing cars
static tCar* getSuccessor(tCar* node);

//INTERNAL FUNCTIONS
//-----
static tCar* insertCarNode(tCar* root, Car* newCar) {
    //checking if new car isn't NULL.
    if (newCar == NULL) {
        return root;
    }
    // in case root doesn't exist it creates one, allocating memory and initializes it.
    if (root == NULL) {
        tCar* newNode = (tCar*)malloc(sizeof(tCar));
        if (newNode == NULL) {
            displayError(1);
            return NULL;
        }
        newNode->p2car = newCar;
        newNode->left = NULL;
    }
}

```

```

        newNode->right = NULL;
        return newNode; //returns the newNode in that case it's the new root.
    }
    //inserts the Car Node to tree based on license.iterates recursively.
    int cmp = strcmp(newCar->nLicense,root->p2car->nLicense);

    if (cmp < 0)
        root->left = insertCarNode(root->left,newCar);
    else if (cmp > 0)
        root->right = insertCarNode(root->right,newCar);
    else
        printf("the car with license %s already in the system\n", newCar->nLicense);

    return root;
}

static Car* searchCarNode(tCar* root, const char* license) {
    //checks if root exists.
    if (root == NULL) {
        return NULL;
    }
    //iterates recursively in the tree and in every iteration trying to compare between
    //the given license and the current iteration root.
    int cmp = strcmp(license,root->p2car->nLicense);
    if (cmp < 0)
        return searchCarNode(root->left,license);
    else if (cmp > 0)
        return searchCarNode(root->right,license);
    else
        return root->p2car; //in case found returns the car node.
}

//getting successor if we delete a node in the car tree.
static tCar* getSuccessor(tCar* node) {
    tCar* temp = node->right;
    while (temp && temp->left)
        temp = temp->left;
    return temp; //returns the successor.
}

static void freeCarNodes(tCar* root) {
    if (root != NULL) {
        freeCarNodes(root->left); // freeing the left kid
        freeCarNodes(root->right); //freeing right kid
        free(root->p2car); // freeing the car
        free(root); //freeing the root
    }
}

////////////////////////////////////
void saveCarsInOrder(const tCar* root, Car** arr, int* index) {
    if (root == NULL)
        return;
    saveCarsInOrder(root->left, arr, index);
    arr[*index] = root->p2car;
}

```

```

    (*index)++;
    saveCarsInOrder(root->right, arr, index);
}

```

//comparing by highest payed

```

int compareByPayed(const void* a, const void* b) {
    if (a == NULL || b == NULL)
        return 0;
    const Car* c1 = *(const Car**)a;
    const Car* c2 = *(const Car**)b;
    if (c2->totalPayed > c1->totalPayed) return 1;
    else if (c2->totalPayed < c1->totalPayed) return -1;
    return 0;
}

```

```

Car* createCar(const char* license, int portType) {
    //license must be 8 digits long
    if (strlen(license) != 8) {
        displayError(19);
        return NULL;
    }
    //if invalid port type returning NULL
    if (portType != FAST && portType != MID && portType != SLOW) {
        displayError(14);
        return NULL;
    }
    // allocate memory for the new car.
    Car* newCar = calloc(1, sizeof(Car));
    if (newCar == NULL) {
        displayError(1);
        return NULL;
    }
    //copies the car information to the new car.
    strcpy(newCar->nLicense, license);
    newCar->portType = portType;
    newCar->totalPayed = 0.0;
    newCar->pPort = NULL;
    newCar->inqueue = 0;
    return newCar; //return the new car
}
//prints the license of the car and the total payment.
void printCar(const Car* c) {
    printf("Car, %s\n", c->nLicense);
    printf("total payed: %.2f\n", c->totalPayed);
}

```

```

int countCarNodes(const tCar* root) {
    if (root == NULL)
        return 0;
    //counting recursively. every node adds 1.
    return 1 + countCarNodes(root->left) + countCarNodes(root->right);
}

```

//counting cars using recursive

```

int countCars(const CarBST* CarTree){
    if (CarTree == NULL) return 0;
    return countCarNodes(CarTree->root);
}

int isCarAvailableForDeletion(const Car* c) {
    if (c == NULL) //if car doesn't exist return 0;
        return 0;
    if (c->pPort != NULL) // if the car pointing to port(means its charging) return 0.
        return 0;
    if (c->inqueue == 1) // if car is in the queue return 0.
        return 0;

    return 1; //returns 1 if available for deletion.
}

tCar* removeCarNode(tCar* node, const char* license) {
    if (node == NULL) //node doesn't exist return NULL.
        return node;
    //comparing given license with the node car license.
    int cmp = strcmp(license,node->p2car->nLicense);
    if (cmp < 0)
        node->left = removeCarNode(node->left, license);
    else if (cmp > 0){
        node->right = removeCarNode(node->right, license);
    }
    //when the comparing succeeded, checking if car available.
    //available - not in queue and not in charge.
    else {
        if (!isCarAvailableForDeletion(node->p2car)) {
            printf("car %s is waiting in line or charging\n", node->p2car->nLicense);
            return node;
        }

        //in case car is available.
        //-----
        //if there is no left kid.
        if (node->left == NULL) {
            tCar* temp = node->right; //saving the right kid to return.
            //releasing the car struct and the node.
            free(node->p2car);
            free(node);
            return temp; // returning the kid to continue connect to the tree.
        }
        //if there is no right kid.
        else if ( node->right == NULL) {
            tCar* temp = node->left; //saving the left kid to return.
            //as mentioned above
            free(node->p2car);
            free(node);
            return temp;
        }

        //if there is two kids, we find the successor.
        tCar* succ = getSuccessor(node);
    }
}

```

```

    tCar* tempRight = node->right;
    tCar* tempLeft = node->left;

    //removing the node where the successor is in from the right kid.
    tempRight = removeCarNode(tempRight, succ->p2car->nLicense);

    //connecting the kids to the successor that replace the current node.
    succ->left = tempLeft;
    succ->right = tempRight;

    //freeing the car struct and the original given node,
    //and returning the successor that will take the original node location in the tree.
    free(node->p2car);
    free(node);
    return succ;
}
//in case the given car isn't available returning the current node.
return node;
}

CarBST* createCarBST() {
    //allocating memory for new binary tree
    //and initialize the field.
    CarBST* CarTree = (CarBST*)malloc(sizeof(CarBST));
    if (!CarTree) {
        displayError(1);
        return NULL;
    }
    CarTree->root = NULL;
    CarTree->totalCars = 0;
    return CarTree;//returning the new tree.
}

int insertCar(CarBST* CarTree, Car* newCar) {
    if (!CarTree || !newCar) return 0;
    if (searchCar(CarTree, newCar->nLicense) != NULL) {
        printf("Car already in the system\n");
        return 0;
    }
    CarTree->root = insertCarNode(CarTree->root, newCar); //running on the tree recursively.
    //update the totalCars field in the tree ADT when inserting a car.
    CarTree->totalCars++;
    return 1;// returning 1 if succeeded.
}

Car* searchCar(CarBST* CarTree, const char* license) {
    if (!CarTree || !license) return NULL;
    return searchCarNode(CarTree->root, license); //search car recursively
}

void freeCarsTree(CarBST* CarTree) {
    if (!CarTree) return;
    //freeing all the car nodes in the tree and after that free the tree itself.

```



```

    freeCarNodes(CarTree->root);
    free(CarTree);
}

```

```

int totalChargeMinutes(Car* car, Date now_time) {
    int startMinutes = DateToMinutes(car->pPort->tin);
    int currentMinutes = DateToMinutes(now_time);

    return currentMinutes - startMinutes;
    // the difference between the current time and the initial charging time of a car in minutes.
}

```

carQueue.h

```

#ifndef CARQUEUE_H
#define CARQUEUE_H

```

```

#include "car.h"
#include "types.h"

```

```

struct Car;
typedef struct Car Car;

```

```

typedef struct carNode {
    Car* p2car;
    struct carNode* next;
}carNode;

```

```

typedef struct {
    carNode* front;
    carNode* rear;
}qCar;

```

```

/**
 * Initializes an empty car queue
 * sets both front and rear pointers to NULL
 * @param q Pointer to the car queue to initialize
 */
void initQueue(qCar* q);

```

```

/**
 * Inserts a new car at the rear of the queue
 * @param q Pointer to the car queue
 * @param car Pointer to the Car to be added to the queue
 */
void enqueueCar(qCar* q, Car* car);

```

```

/**
 * Removes and returns the car at the front of the queue
 * @param q Pointer to the car queue
 * @return Pointer to the Car at the front, or NULL if queue's empty
 */
Car* dequeueCar(qCar* q);

```

```

/**
 * check whether the queue's empty
 * @param q Pointer to the car queue
 * @return 1 if the queue is empty, 0 otherwise
 */
int isEmptyQueue(const qCar* q);

/**
 * Prints all cars currently in the queue in order
 * @param q Pointer to the car queue
 */
void printQueueCars(const qCar* q);

/**
 * Returns the position of a specific car in queue(first car is at position 1)
 * @param q Pointer to the car Queue
 * @param car the Pointer to the car for search
 * @return The car's position in the queue, or -1 if not found
 */
int getQueuePosition(const qCar* q, Car* car);

/**
 * Retrieves the next car in the queue that matches the specific type.
 * @param carQueue Ponter to the car queue struct.
 * @param type Desired port type to match
 * @return Pointer to a Car object if found, or NULL if:
 * - the queue is empty
 * - No car in the queue matches the given port type.
 *
 * This function iterated through the queue from front to back,
 * checking each car's port type. If a car matching the desired type
 * is found, a pointer to it is returned, If none match, NULL is returned.
 */
Car* getNextCarInQueue(qCar* carQueue, portType type);

#endif //CARQUEUE_H

```

carQueue.c

```

#include <stdio.h>
#include <stdlib.h>
#include "car.h"
#include "carQueue.h"

void initQueue(qCar* q) {
    //initializes queue ADT.
    if (q != NULL) {
        q->front = NULL;
        q->rear = NULL;
    }
}

void enqueueCar(qCar* q, Car* car) {
    carNode* newCar = (carNode*)malloc(sizeof(carNode)); //allocate memory for car node in queue
    if (newCar == NULL) {

```

```

        printf("Memory allocation failed in enqueue()\n");
        return;
    }
    newCar->p2car = car;
    newCar->next = NULL;

    //in case rear queue is NULL(means the queue is empty) both front and rear points to the new
    car.
    if (q->rear == NULL) {
        q->front = q->rear = newCar;
    }
    //else there's cars in the queue so adding the node to rear of queue.
    else {
        q->rear->next = newCar; // linking the previous node to the new node.
        q->rear = newCar; // updating the rear pointer to the new node.
    }
    car->inqueue = 1; //car is in queue.
}

Car* dequeueCar(qCar* q) {
    //if queue is empty or not initialize.
    if (q == NULL || q->front == NULL || q->rear == NULL) return NULL;

    Car* temp = q->front->p2car; // temporary pointer to the car in the front.
    carNode* NodeToFree = q->front; // pointer to the node that we will free.

    q->front = q->front->next;

    //if the queue empty after dequeue the rear=NULL.
    if (q->front == NULL) {
        q->rear = NULL;
    }
    temp->inqueue = 0; //car is out of queue.

    free(NodeToFree); // freeing the node.

    return temp; // returning a pointer to the dequeued car.
}

int isEmptyQueue(const qCar* q) {
    return (q == NULL || q->front == NULL);
}

//printing car that in queue.
void printQueueCars(const qCar* q) {
    if (isEmptyQueue(q)) {
        printf("Queue is empty\n");
        return;
    }
    carNode* temp = q->front;
    int pos = 1;
    printf("Cars in queue:\n");
    printf("-----\n");

    while (temp != NULL) {

```

```

        Car* car = temp->p2car;
        printf("Car %d: ", pos);
        printf("License: %s ", car->nLicense);
        printf("Type: %s \n", portTypeToStr(car->portType));
        pos++;
        temp = temp->next;
    }
    printf("-----\n");
}

int getQueuePosition(const qCar* q, Car* car) {
    if (q == NULL || q->front == NULL || car == NULL) {
        return -1;
    }
    carNode* temp = q->front;
    int pos = 1;
    //running in the queue from front to back and when the given car found returns her position.
    while (temp != NULL) {
        if (temp->p2car == car) {
            return pos;
        }
        temp = temp->next;
        pos++;
    }
    return -1;
}

Car* getNextCarInQueue(qCar* carQueue, portType type) {
    if (isEmpty(carQueue)) return NULL; // checks if queue is empty
    carNode* temp = carQueue->front; //temporary pointer to front of the queue
    //passing on all the
    while (temp) {
        //if p2car isn't NULL and the port type of the car in queue equal to type, then it returns the car
        //and connect it to port
        if (temp->p2car && temp->p2car->portType == type) {
            return temp->p2car;
        }
        temp = temp->next;
    }

    return NULL; //no compatible car in queue
}

```

date.h

```

#ifndef DATE_H
#define DATE_H

```

```

typedef struct {
    int Year; //4 digit year in range 1990-2025
    int Month; // 1-12
    int Day; //1-31 depending on the month
    int Hour; //0-23(24 hours format)
    int Min; //0-59

```

```

}Date;

/**
 * Creates and returns a Date object with specified values.
 * no validation performed; use isValidDate to check.
 * @param year
 * @param month
 * @param day
 * @param hour
 * @param min
 * @return
 */
Date createDate(int year, int month, int day, int hour, int min);

/**
 * Returns the current system date and time as a Date struct.
 * @return Date representing the current time
 */
Date getCurrentDate();

/**
 * Converts the date to minutes.
 * @param date The Date to convert to minutes
 * @return The number of minutes
 *
 * This function multiply the: (years, months, days, hours) by the amount of minutes in them
 * and sum up them.
 */
int DateToMinutes(Date date);

#endif //DATE_H

```

date.c

```

#include "date.h"
#include <stdio.h>
#include <stdbool.h>
#include <time.h>

Date createDate(int year, int month, int day, int hour, int min) {
    Date date;
    date.Year = year;
    date.Month = month;
    date.Day = day;
    date.Hour = hour;
    date.Min = min;
    return date;
}

Date getCurrentDate() {
    Date date;
    time_t now = time(NULL);
    struct tm* tm_info = localtime(&now);

    date.Year = tm_info->tm_year + 1900;

```

```

    date.Month = tm_info->tm_mon + 1;
    date.Day = tm_info->tm_mday;
    date.Hour = tm_info->tm_hour;
    date.Min = tm_info->tm_min;

    return date;
}

int DateToMinutes(Date date) {
    int years_to_minutes = date.Year * 525600; //multiply years by minutes in a year
    int months_to_minutes = date.Month * 43800; //multiply months by minutes in a month
    int days_to_minutes = date.Day * 1440; // multiply days by minutes in a day
    int hours_to_minutes = date.Hour * 60; // multiply hours by minutes in an hour
    return years_to_minutes + months_to_minutes + days_to_minutes + hours_to_minutes +
    date.Min;
}

```

files.h

```

#ifndef FILES_H
#define FILES_H

#include <stdio.h>
#include "Station.h"
#include "car.h"

/**
 * load files to ADT'S
 * @param CarTree Pointer to car BST
 * @param stationTree Pointer to station BST
 */
void loadFiles(CarBST* CarTree, StationBST* stationTree);

/**
 * Load station.txt file information to station BST
 * @param stationTree Pointer to station BST
 */
void loadStations(StationBST* stationTree);

/**
 * Load station.txt file information to station BST
 * @param stationTree Pointer to station BST
 */
void loadPorts(CarBST* CarTree, StationBST* stationTree);

/**
 * Load port.txt file information to one way linked ports lists
 * @param CarTree Pointer to car BST
 * @param stationTree Pointer to station BST
 */
void loadCars(CarBST* CarTree, StationBST* stationTree);

/**
 * Load car.txt file information to car BST
 * @param stationTree Pointer to station BST
 * @param CarTree Pointer to car BST

```

```

*/
void loadQueue(StationBST* stationTree, CarBST* CarTree);

/**
 * Updates all files from ADT'S.
 * @param stationTree Pointer to the Station BST.
 * @param CarTree Point to the Car BST.
 * Update the files after every action taken. Handles every ADT has it should.
 */
void updateFiles(StationBST* stationTree, CarBST* CarTree);

/**
 * Writes the Cars details to the given file.
 * @param filename Pointer to the name of the Cars file.
 * @param CarTree Pointer to the Car BST
 * @param stationTree Pointer to the Station BST.
 *
 * This function iterates in the Car tree and write the details of each Car in the asked order.
 */
void saveCarsToFile(const char* filename, CarBST* CarTree, StationBST* stationTree);

/**
 * Writes the Stations details to the given file.
 * @param filename Pointer to the name of the Stations file.
 * @param stationTree Pointer to the Station BST.
 *
 * This function iterates in the Station tree and write the details of each Station,in the asked order.
 */
void saveStationsToFile(const char* filename, StationBST* stationTree);

/**
 * Writes the Ports details to the given file.
 * @param filename Pointer to the name of the Ports file.
 * @param stationTree Pointer to the Station BST.
 *
 * This function iterates in the station tree, and every station is pointing to the portlist field,
 * and runs on the portlist linked list and writes the details of each Port in the Portlist, in the asked
 order.
 */
void savePortsToFile(const char* filename, StationBST* stationTree);

/**
 * Writes the details od the Cars waiting in queue.
 * @param filename Pointer to the name of the Queue file.
 * @param stationTree Pointer to the Station BST.
 *
 * This function iterates in the station tee, and every station is pointing to carQueue field,
 * and runs on the queue from front to back and writes the details of each Car in queue.
 */
void saveQueuesToFile(const char* filename, StationBST* stationTree);

#endif

```

files.c

```
#include "files.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "Station.h"
#include "car.h"
#include "carQueue.h"
#include "helper_functions.h"
#include "types.h"

#define LINE_SIZE 256
#define CAR_LINE_SIZE 128
#define NAME_SIZE 100 //assuming the max length of a station name is 100
#define NO_LICENSE "-1"

void loadStations(StationBST* stationTree) {
    FILE* fp = fopen("stations.txt", "r");// open file to read from
    if (fp == NULL) { // if file is doesn't exist return;
        printf("Error opening file\n");
        return;
    }
    char line[LINE_SIZE];
    fgets(line, sizeof(line), fp); // Skip header
    //starting to go over the file.
    while (fgets(line, sizeof(line), fp)) {
        int id, numPorts;
        char name[NAME_SIZE];
        double x,y;
        // reading information in order of what's written into matching variables
        int count = sscanf(line, "%d,%99[^\,],%d,%f,%f", &id,name,&numPorts, &x, &y);
        //in case it was loading fine we create a station with the information we loaded.
        if (count == 5) {
            Coord coord = {x, y};
            Station* newStation = createStation(id, name, coord);
            //if the creation was smooth we can insert the station to the station tree.
            if (newStation) {
                insertStation(stationTree, newStation);
            }
        } else {
            printf("Parsing error: %s", line);
        }
    }
    fclose(fp); //closing the file.
}

void loadCars(CarBST* CarTree, StationBST* stationTree) {
    FILE* fp = fopen("Cars.txt", "r");//open file for reading.
    if (fp == NULL) {
        printf("couldn't open Cars.txt\n");
        return;
    }
}
```



```

char line[LINE_SIZE];
fgets(line, sizeof(line), fp); // skip header

//starting to go over the file.
while (fgets(line, sizeof(line), fp)) {
    char license[9];
    char typeStr[9];
    double totalPayed;
    int stId, portNum, inQueue;

    // reading information in order of what's written into matching variables
    int count = sscanf(line,
"%[^,],%[^,],%lf,%d,%d,%d", license, typeStr, &totalPayed, &stId, &portNum, &inQueue);

    //if it didn't load all of the information.
    if (count != 6) {
        printf("format error: %s", line);
        continue;
    }

    portType type = portTypeFromStr(typeStr); //converting type string to portType enum.

    //creating a car with the information we loaded.
    Car* newCar = createCar(license, type);
    if (!newCar) continue;

    //initializes rest of the information.
    newCar->totalPayed = totalPayed;
    newCar->inqueue = inQueue;
    newCar->pPort = NULL;

    //inserting the new car to Car tree.
    insertCar(CarTree, newCar);
}
fclose(fp); //closing the file
}

void loadPorts(CarBST* CarTree, StationBST* stationTree){
    FILE* fp = fopen("Ports.txt", "r"); // opening file for reading
    if (fp == NULL) {
        printf("couldn't open Ports.txt\n");
        return;
    }
    char line[LINE_SIZE];
    fgets(line, sizeof(line), fp); //skip header.
    //starting to go over the file.
    while (fgets(line, sizeof(line), fp)){
        int stId, portNum, status;
        char license[9];
        char typeStr[9];
        Date tin = {0};

        // reading information in order of what's written into matching variables

```

```

    int count = sscanf(line, "
%d,%d,%[^,],%d,%d,%d,%d,%d,%d,%8s",&stId,&portNum,typeStr,&status,&tin.Year,&tin.Month,&ti
n.Day,&tin.Hour,&tin.Min,license);

    //if it didn't load all of the information
    if (count != 10) {
        printf("format error: %s", line);
        continue;
    }
    portType type = portTypeFromStr(typeStr);//converting string type to portType enum.
    if (type == -1){
        printf("invalid port type: %s\n",license);
        continue;
    }

    //creating port with the information we loaded
    Port* newPort = createPort(portNum, type);
    if (!newPort) continue;

    //initializes rest of the information.
    newPort->status = status;
    newPort->tin = tin;

    //if the license we loaded is different from '-1'
    if (strcmp(license,NO_LICENSE) != 0) {
        //searching car and connecting it to port from both sides(port to car, car to port).
        Car* c = searchCar(CarTree, license);
        if (c) {
            newPort->p2car = c;
            c->pPort = newPort;
        }
    }
    //after creating the port we adds it to the station, searching by station id.
    Station* st = searchStationById(stationTree,stId);
    if (st) {
        addPortToStation(st, newPort);// adding port to station...
    }
    else {
        //in case the station was not found, print error and freeing the new port
        printf("Station %d not found\n", stId);
        free(newPort);
    }
}
fclose(fp);//closing the file.
}

```

```

void loadQueue(StationBST* stationTree, CarBST* CarTree){
    FILE* fp = fopen("LineOfCars.txt", "r"); // open file to read.
    if (fp == NULL) {
        printf("couldn't open LineOfCars.txt\n");
        return;
    }
    char line[LINE_SIZE];
    fgets(line, sizeof(line), fp); //skip header
}

```

```

//starting to go over the file.
while (fgets(line, sizeof(line), fp)) {
    int stld;
    char license[9];
    //reading information in order of what's written into matching variables.
    int count = sscanf(line, "%8[^,],%d", license, &stld);

    if (count != 2) {
        printf("format error: %s", line);
        continue;
    }
    //looking if the car is in the car tree
    Car* c = searchCar(CarTree, license);
    if (!c) {
        printf("Car %s not found\n", license);
        continue;
    }
    //looking for the station by id that was loaded.
    Station* st = searchStationById(stationTree, stld);
    if (!st) {
        printf("Station %d not found\n", stld);
        continue;
    }
    //enqueueing the car into the station car queue.
    //updating nCars field in the station and that the car is now in queue.
    enqueueCar(st->carQueue, c);
    c->inqueue = 1;
    st->nCars++;
}
fclose(fp);
}

//takes all the load function and use them.
void loadFiles(CarBST* CarTree, StationBST* stationTree) {
    loadStations(stationTree);
    loadCars(CarTree, stationTree);
    loadPorts(CarTree, stationTree);
    loadQueue(stationTree, CarTree);
}

static void writeCarToFile(FILE* fp, tCar* node, StationBST* stationTree) {
    if (!node) return;
    //writing recursively to file(car BST).
    writeCarToFile(fp, node->left, stationTree);

    Car* car = node->p2car; //initialize a temporary car for each iteration in the tree.
    int stld = 0;
    int portNum = 0;

    //pPort can be NULL
    if (car->pPort) {
        //if it isn't NULL, we are searching the information by the port the car plug to.
        stld = findStationIdByPort(stationTree, car->pPort);
        portNum = car->pPort->num;
    }
}

```

```

    }
    //check if car is not connected but sits in station queue
    else if (car->inqueue == 1) {
        Station* station = findStationByCarInQueue(stationTree->root, car);
        if (station) {
            stld = station->id;
        }
    }
    //write the information to the file in order.
    fprintf(fp, "%s,%s,%2lf,%d,%d,%d\n", car->nLicense, portTypeToStr(car->portType), car->totalPaid, stld, portNum, car->inqueue);

    writeCarToFile(fp, node->right, stationTree);
}

void saveCarsToFile(const char* filename, CarBST* CarTree, StationBST* stationTree) {
    FILE* fp = fopen(filename, "w"); //deleting the previous file and opening a new one for read & write.
    if (fp == NULL) {
        printf("couldn't open file %s\n", filename);
        return;
    }
    //writing header.
    fprintf(fp, "License,PortType,TotalPaid,StationID,PortNumber,InQueue\n");
    writeCarToFile(fp, CarTree->root, stationTree); //writing car information into file.
    fclose(fp);
}

static void writeStationToFile(FILE* fp, Station* node) {
    if (!node) return;
    //going over the station tree recursively
    //and write information to file in every node iteration.
    writeStationToFile(fp, node->left);
    //writing in order of the file.(like loading from...)
    fprintf(fp, "%d,%s,%d,%2lf,%2lf\n", node->id, node->name, node->nPorts, node->coord.x, node->coord.y);
    writeStationToFile(fp, node->right);
}

void saveStationsToFile(const char* filename, StationBST* stationTree) {
    FILE* fp = fopen(filename, "w");
    if (fp == NULL) {
        printf("couldn't open file %s\n", filename);
        return;
    }
    //writing header.
    fprintf(fp, "ID,StationName,NumOfPorts,CoordX,CoordY\n");
    writeStationToFile(fp, stationTree->root); //^
    fclose(fp);
}

static void writePortsToFile(FILE* fp, Station* node) {
    if (!node) return;
    writePortsToFile(fp, node->left);
    //going over the station tree and in every iteration station point to portlist.

```

```

Port* temp = node->portsList->head;
//running on the linked list portlist.
while (temp) {
    char license[9] = "-1";
    if (temp->p2car)
        strcpy(license, temp->p2car->nLicense);
    //prints the port information that should be in the txt file.
    fprintf(fp, "%d,%d,%s,%d,%d,%d,%d,%d,%d,%s\n", node->id, temp-
>num, portTypeToStr(temp->portType), temp->status
, temp->tin.Year, temp->tin.Month, temp->tin.Day, temp->tin.Hour, temp->tin.Min, license);
    temp = temp->next;
}
writePortsToFile(fp, node->right);
}

```

```

void savePortsToFile(const char* filename, StationBST* stationTree) {
    FILE* fp = fopen(filename, "w");
    if (fp == NULL) {
        printf("couldn't open file %s\n", filename);
        return;
    }
    // writing header
    fprintf(fp, "StationID,PortNumber,Type,Status,Year,Month,Day,Hour,Min,License\n");
    writePortsToFile(fp, stationTree->root);
    fclose(fp);
}

```

//same as write ports but just the station points to carQueue field.

```

static void writeQueuesToFile(FILE* fp, Station* node){
    if (!node) return;
    writeQueuesToFile(fp, node->left);
    carNode* temp = node->carQueue->front;
    //running on the queue and print the information.
    while (temp) {
        fprintf(fp, "%s,%d\n", temp->p2car->nLicense, node->id);
        temp=temp->next;
    }
    writeQueuesToFile(fp, node->right);
}

void saveQueuesToFile(const char* filename, StationBST* stationTree) {
    FILE* fp = fopen(filename, "w");
    if (fp == NULL) {
        printf("couldn't open file %s\n", filename);
        return;
    }
    //writing header.
    fprintf(fp, "License,StationID\n");
    writeQueuesToFile(fp, stationTree->root);
    fclose(fp);
}

//takes all the update files and use them.
void updateFiles(StationBST* stationTree, CarBST* CarTree) {
    saveStationsToFile("Stations.txt", stationTree);
    saveCarsToFile("Cars.txt", CarTree, stationTree);
    savePortsToFile("Ports.txt", stationTree);
}

```

```

    saveQueuesToFile("LineOfCars.txt", stationTree);
}

```

helper_functions.h

```

#ifndef HELPER_FUNCTIONS_H
#define HELPER_FUNCTIONS_H

#include "Station.h"

Station* searchStation(const StationBST* tree, int searchType, void* data);
/**
 * select a charging station by either ID or name.
 * @param stationTree Pointer to the station BST
 * @return Pointer to the selected Station if found
 * NULL if user cancels, station not found, or stationTree is NULL.
 */
Station* getStationByIdOrName(StationBST* stationTree);
/**
 * select a charging station by ID
 * @param stationTree Pointer to the station BST
 * @return Pointer to the selected Station if found
 * NULL if user cancels, station not found, or stationTree is NULL.
 */
Station* getStationById(StationBST* stationTree);
void displayError(int errorCode);

```

```

#endif //HELPER_FUNCTIONS_H

```

helper_functions.c

```

#define _CRT_SECURE_NO_WARNINGS
#include "helper_functions.h"
#include "Station.h"
#include <stdio.h>

#define MAX_ST_NAME 100

typedef Station* (*SearchFunc)(const StationBST*, void*);

Station* searchById(const StationBST* tree, void* data) {
    int id = *(int*)data;
    return searchStationById(tree, id);
}
Station* searchByName(const StationBST* tree, void* data) {
    return searchStationByName(tree, (char*)data);
}
Station* searchByCoordinate(const StationBST* tree, void* data) {
    return searchStationByDistance(tree, (Coord*)data);
}
//array of pointers to search station function.
SearchFunc searchFunctions [3] = {searchById, searchByName, searchByCoordinate};

//function that gets the type of search station and searching in match of it.
Station* searchStation(const StationBST* tree, int searchType, void* data) {
    if (!tree || !data) return NULL;

```

```

    if (searchType < 0 || searchType > 2) return NULL;
    return searchFunctions[searchType](tree, data);
}
Station* getStationByIdOrName(StationBST* stationTree) {
    if (!stationTree) {
        displayError(7);
        return NULL;
    }
    int choice;
    Station* station = NULL;
    do {
        printf("Search station by: ID-1, Name- 2? : ");
        if (scanf("%d", &choice) != 1) {
            printf("Invalid input. please enter 1 or 2.\n");
            while (getchar() != '\n');
            continue;
        }
        if (choice != 1 && choice != 2)
            printf("Invalid input. please enter 1 or 2.\n");
    } while (choice != 1 && choice != 2);
    do {
        if (choice == 1) {
            int stId;
            printf("Enter station ID or -1 to cancel: ");
            if (scanf("%d", &stId) != 1) {
                printf("Invalid station ID. please enter an ID\n");
                while (getchar() != '\n');
                continue;
            }
            if (stId == -1) return NULL;
            station = searchStation(stationTree, 0, &stId);
        }
        else {
            char name[MAX_ST_NAME];
            printf("Enter station name or exit to cancel: ");
            scanf(" %[^\\n]", name);
            if (strcmp(name, "exit") == 0) return NULL;
            station = searchStation(stationTree, 1, name);
        }
        if (!station) {
            displayError(3);
            return NULL;
        }
    } while (!station);

    return station;
}
Station* getStationById(StationBST* stationTree) {
    if (!stationTree) {
        displayError(7);
        return NULL;
    }
    int stId;
    Station* station = NULL;

```

```

do {
    printf("Enter station ID or -1 to cancel: ");
    if (scanf("%d", &stId) != 1) {
        printf("Invalid input. please enter a number\n");
        while (getchar() != '\n');
        continue;
    }
    station = searchStation(stationTree, 0, &stId);
    if (!station) {
        displayError(3);
        return NULL;
    }
} while (!station);
return station;
}

```

//prints errors by code number

```

void displayError(int errorCode) {

```

```

    switch (errorCode) {
        case 1:
            printf("Error: Memory allocation failed.\n");
            break;
        case 2:
            printf("Error: Failed to open file..\n");
            break;
        case 3:
            printf("Error: Station not found.\n");
            break;
        case 4:
            printf("Error: Station cannot be removed (cars charging or in queue).\n");
            break;
        case 5:
            printf("Error: Car not found.\n");
            break;
        case 6:
            printf("Error: Invalid input format.\n");
            break;
        case 7:
            printf("Error: Tree not initialized.\n");
            break;
        case 8:
            printf("Error: Invalid choice.\n");
            break;
        case 9:
            printf("Error: Car already charging.\n");
            break;
        case 10:
            printf("Error: Car already in queue.\n");
            break;
        case 11:
            printf("Error: No available port of required type.\n");
            break;
        case 12:

```



```

        printf("Error: Assignment failed - car could not be connected.\n");
        break;
    case 13:
        printf("Error: Car is not currently charging.\n");
        break;
    case 14:
        printf("Error: Invalid port type.\n");
        break;
    case 15:
        printf("Error: Failed to create new port.\n");
        break;
    case 16:
        printf("Error: Failed to exit system properly.\n");
        break;
    case 17:
        printf("Error: Failed to remove port.\n");
        break;
    case 18:
        printf("Error: No out-of-order ports found.\n");
        break;
    case 19:
        printf("Error: License must be exactly 8 digits.\n");
        break;
    default:
        printf("Unknown error.\n");
        break;
    }
}

```

menu.h

```

#ifndef MENU_H
#define MENU_H

/**
 * Prints all menu actions.
 */
void Menu();

void clear();

#endif //MENU_H

```

menu.c

```

//
// Created by asafi on 7/9/2025.
//
#include "menu.h"
#include <stdio.h>
#include <stdlib.h>

void Menu() {
    printf("--Welcome to EasyCharge management platform--\n");
    printf("====MainMenu====\n");
    printf("1.Locate nearest station\n");
}

```

```

    printf("2.Charge car\n");
    printf("3.Check car status\n");
    printf("4.Stop charge\n");
    printf("5.Display all stations\n");
    printf("6.Display cars at station\n");
    printf("7.Report of stations' statistics\n");
    printf("8.Display Top customers\n");
    printf("9.Add new port\n");
    printf("10.Release Charging Ports\n");
    printf("11.Remove out of order ports\n");
    printf("12.Remove customer\n");
    printf("13.Close station\n");
    printf("0.Exit System\n");
    printf("Enter your choice: ");
}

```

```

void clear() {
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
}

```

menu.functions.h

```

#ifndef MENU_FUNCTIONS_H
#define MENU_FUNCTIONS_H
#include "Station.h"

```

```

/**
 * Finds the closest station based on user input coordinates.
 *
 * @param stationTree Pointer to station tree.
 *
 * Asks for location input (x and y) and find the closest station.
 * Prints: station name, ID, distance from the location input, number of ports and port details (type
and status)
 * and the number of cars waiting in line.
 */

```

```

void locNearSt(const StationBST *stationTree);

```

```

/**
 * Connecting a car to a charger at selected station.
 *
 * @param stationTree Pointer to the station tree.
 * @param carTree Pointer to the car tree.
 *
 * The function lets user choose station by ID or name, then input the car license.
 * if the car isn't found in the system, the user can enter its details and add it to the tree.
 * if there's a port available, the car begins charging. Otherwise, it's added to the station queue.
 */

```

```

void chargeCar(StationBST* stationTree, CarBST* carTree);

```

```

/**
 * Displays current status of a car: charging, in queue, or idle.
 *

```

```

* @param stationTree Pointer to the station tree.
* @param carTree Pointer to the car tree.
*
* Asks the use to enter a car license and searches for it in the car tree.
* if the car is charging, displays the station, port, and time elapsed.
* if the car is waiting in queue, shows the station and its queue position.
* Otherwise, informs the user the car is not charging or in queue.
*/
void checkCarStatus(StationBST* stationTree, CarBST* carTree);

/**
* Stops charging for a specific car and assigns the next car from queue if available.
*
* @param stationTree Pointer to the station tree.
* @param carTree Pointer to the car tree.
*
* Calculates the cost based on charging duration, Update car and port status.
*/
void stopCharge(StationBST* stationTree, CarBST* carTree);

/**
* Prints akk station with summary details.
*
* @param stationTree Pointer to the station tree.
*
* going through the station tree and displays:
* name, ID, list of ports, number of cars charging and num of cars in queue.
*
*/
void dispAllSt(StationBST* stationTree);

/**
* Display all cars at a specific station.
*
* @param stationTree Pointer to the station tree.
*
* Asks for station name or ID, then shows all charging and queued cars with relevant details.
*/
void dispCarsAtSt(StationBST* stationTree);

/**
* generates report on a selected station.
*
* Shows utilization rate %, faulty port ratio %, and load balance.
*
* @param stationTree Pointer to the station tree.
*/
void reportStStat(StationBST* stationTree);

/**
* Displays top 5 cars with highest total payments.
*
* Traverses the car tree, collects all cars into an array,
* and sorts them by their totalPayed field.
* Prints license and full details of the top 5 paying customers.

```

```

*
* Allocates memory for sorting process and frees it afterward.
* shows an error if memory allocation fails or if no cars exist.
*
* @param CarTree Pointer to the car tree.
*/
void dispTopCustomers(const CarBST* CarTree);

/**
* Adds a new charging port to a specific station.
*
* Asks user to choose station and port type.
* creates new port and connects a car from queue if available.
*
* @param stationTree Pointer to the station tree.
*/
void addNewPort(StationBST* stationTree);

/**
* Releases any car that has been charging for over 10 hours at a given station.
*
* Disconnects overdue cars, calculates payment, and assigns queued cars to free ports if possible.
*
* @param stationTree
*/
void releasePorts(StationBST* stationTree);

/**
* Removes "out-of-order" charging port from selected station.
*
* Lists all the faulty ports in station, then removes based on user input.
*
* @param stationTree Pointer to the station tree.
*/
void remOutOrderPort(StationBST* stationTree);

/**
* Deletes a car from the system.
*
* Car can only be removed if it's not in queue or currently charging, free memory.
* @param CarTree Pointer to the car tree.
*/
void remCustomer(CarBST* CarTree);

/**
* Closes a selected station permanently.
* Station can only be removed if there's no cars in queue or in charge int the station.
* Removes station from tree and frees memory.
*
* @param stationTree Pointer to the station tree.
*/
void closeSt(StationBST *stationTree);

/**
* Safely exits the system and saves all data.

```

```

*
* Saves current data to files(stations, ports, cars, queue) Free all memory and confirms shutdown.
*
* @param stationTree Pointer to the station tree.
* @param carTree Pointer to the car tree.
*/
void exitSys(StationBST* stationTree, CarBST* carTree);

#endif //MENU_FUNCTIONS_H

```

menu.functions.c

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
#include <time.h>
#include "Station.h"
#include "menu_functions.h"
#include "car.h"
#include "helper_functions.h"
#include "carQueue.h"
#include "files.h"
#include "date.h"

#define LICENSE_SIZE 9
#define MAX_ST_NAME 100

//ACTION 1
void locNearSt(const StationBST *stationTree) {
    //Find nearest station by coordinates
    Coord userCoord = {0,0};
    while (1) {
        printf("Enter coordinates(x,y) separated by commas: ");
        if (scanf("%lf,%lf", &userCoord.x, &userCoord.y) == 2) break;
        printf("Invalid input, Please enter two numbers seperated by commas\n");

        while (getchar() != '\n');
    }
    printf("\n");
    Station* nearest = searchStation(stationTree, 2, &userCoord); // search station by coordinates.
    if (!nearest) {
        displayError(3);
        return;
    }
    //calculates the distance from input loc to the near st loc
    double distance = calculateDistance(userCoord, nearest->coord);

    //ports details at nearest station.
    int totalPorts = nearest->nPorts;
}

```

```

int activePorts = 0;
int busyPorts = 0;
int slow = 0, mid = 0, fast = 0;
int waiting = nearest->nCars;

Port* temp = nearest->portsList->head;
//counting amount of ports that not faulty and the ports in use.
while (temp) {
    if (temp->status != OUT_OF_ORDER) {
        activePorts++;
        if (temp->status == OCCUPIED) {
            busyPorts++;
        }
    }
    //counting amount of port of certain type.
    switch (temp->portType) {
        case SLOW: slow++; break;
        case MID: mid++; break;
        case FAST: fast++; break;
        default: break;
    }
    temp = temp->next;
}

printf("The nearest station: %s\n", nearest->name);
printf("Distance: %0.2f\n", distance);
printf("Total ports: %d\n", totalPorts);
printf("Active ports: %d\n", activePorts);
printf("Busy ports (with car): %d\n", busyPorts);
printf("Cars waiting in queue: %d\n", waiting);
printf("Port types:\n");
printf("  SLOW: %d\n", slow);
printf("  MID : %d\n", mid);
printf("  FAST: %d\n", fast);
}

//Action 2
void chargeCar(StationBST* stationTree, CarBST* carTree) {
    if (!stationTree || !carTree) {
        displayError(7);
        return;
    }
    Station* station = getStationByIdOrName(stationTree);
    if (!station) return;

    //input of car license
    char license[LICENSE_SIZE];
    Car* car = NULL;

    while (!car) {
        printf("Enter license(8 characters) or 'exit' to cancel: ");
        scanf(" %s", license);
        if (strcmp(license, "exit") == 0) return;

        if (strlen(license) != 8) {
            displayError(19);

```

```

        continue;
    }
    //checks if the license inputs is valid and only digits.
    int valid = 1;
    for (int i = 0; i < strlen(license); i++) {
        if (!isdigit(license[i])) {
            valid = 0;
            break;
        }
    }

    if (!valid) {
        displayError(19);
        continue;
    }

    car = searchCar(carTree,license);
    if (!car) {
        printf("Car %s not found. Please enter its details:\n",license);

        char typeStr[10];
        portType type;
        //creation of car that's not exists
        do {
            printf("Enter port type(SLOW/MID/FAST): ");
            scanf("%9s", typeStr);
            type = portTypeFromStr(typeStr);
            if (type == -1)
                printf("Invalid port type. Try again.\n");
        }while (type == -1);
        car = createCar(license,type);
        if (!car) {
            printf("Failed to create car. Try again\n");
            continue;
        }
        insertCar(carTree,car);
        printf("Car %s added to system.\n", license);
    }
}

//if car isn't connected to port.
if (car->pPort) {
    displayError(9);
    return;
}

//if car is in queue.
if (car->inqueue == 1) {
    displayError(10);
    return;
}

//if available port isn't found in station
Port* port = findAvailablePortInStation(station,car->portType);
if (!port) {
    displayError(11);
    enqueueCar(station->carQueue,car);
}

```

```

        car->inqueue = 1;
        station->nCars++;
        printf("Car %s has entered the queue at station %s.\n",car->nLicense,station->name);
        return;
    }
    //assign car to port and updating time of charging.
    if (assignCar2port(car, port)) {
        port->tin = getCurrentDate();
        printf("Car %s is now charging at port %d, in station %s.\n",car->nLicense,port->num,station-
>name);
        return;
    }else {
        displayError(12);
    }
}
//Action 3
void checkCarStatus(StationBST* stationTree, CarBST* carTree) {
    if (!stationTree || !carTree) {
        displayError(7);
        return;
    }
    char license[LICENSE_SIZE];
    printf("Enter license(8 characters) or 'exit' to cancel: ");
    scanf(" %8s", license);
    if (strcmp(license, "exit") == 0) return;
    Car* car = searchCar(carTree,license);
    if (!car) {
        displayError(5);
        return;
    }

    int stld = -1;
    //in case car is connected to port.
    if (car->pPort) {
        stld = findStationIdByPort(stationTree,car->pPort);
        Station* station = searchStation(stationTree,0,&stld);
        if (!station) {
            printf("Car is charging, but station data is missing.\n");
            return;
        }
        //calculates the charge time in minutes.
        Date now = getCurrentDate();
        int totalMinutes = totalChargeMinutes(car,now);
        if (totalMinutes < 0) totalMinutes = 0;

        printf("Car %s is charging at station \"%s\", port #%d.\n",car->nLicense,station->name, car-
>pPort->num);
        printf("Charging time: %d minutes.\n",totalMinutes);
        return;
    }

    //handles the situation when the car is in queue.
    if (car->inqueue == 1){
        Station* QueueStation = findStationByCarInQueue(stationTree->root,car);
        if (!QueueStation) {

```



```

        printf("Car %s marked in queue but Station was not found.\n", license);
        return;
    }
    int pos = getQueuePosition(QueueStation->carQueue, car);
    if (pos != -1) {
        printf("Car %s is in queue at station \"%s\"\n", car->nLicense, QueueStation->name);
        printf("Position in %s charging queue: %d.\n", portTypeToStr(car->portType), pos);
    } else {
        printf("Car %s marked in queue but not found in station queue.\n", car->nLicense);
    }
    return;
}
printf("Car %s is neither charging nor in queue.\n", car->nLicense);
}
//Action 4
void stopCharge(StationBST* stationTree, CarBST* carTree) {
    if (!stationTree || !carTree) {
        displayError(7);
        return;
    }
    char license[LICENSE_SIZE];
    printf("Enter license(8 characters) or 'exit' to cancel: ");
    scanf(" %8s", license);
    if (strcmp(license, "exit") == 0) return;
    Car* car = searchCar(carTree, license);
    if (!car) {
        displayError(5);
        return;
    }
    if (!car->pPort) {
        displayError(13);
        return;
    }
    //searching for the station the car has found at.
    int stId = findStationIdByPort(stationTree, car->pPort);
    Station* station = searchStation(stationTree, 0, &stId);
    if (!station) {
        displayError(3);
        return;
    }
    Date now = getCurrentDate();
    //calculates total time car is charged in minutes.
    int totalMinutes = totalChargeMinutes(car, now);
    if (totalMinutes < 0) totalMinutes = 0;

    //determines the payment based on the time.
    double price = totalMinutes * 1.2;
    car->totalPaid += price;

    printf("Car %s finished charging at station \"%s\", port # %d.\n", car->nLicense, station->name,
car->pPort->num);
    printf("Charging time: %d minutes. price to pay: %.2lf ILS.\n", totalMinutes, price);

    //disconnecting car from port
    car->pPort->tin = createDate(0,0,0,0,0);
}

```

```

car->pPort->status = FREE;
car->pPort->p2car = NULL;
car->pPort = NULL;
station->nCars--;

//handles if there's a matching car to the port type in queue.
Car* nextCar = getNextCarInQueue(station->carQueue, car->portType);
if (nextCar) {
    Port* port = findAvailablePort(station->portsList, nextCar->portType);
    if (port) {
        nextCar->pPort = port;
        port->p2car = nextCar;
        port->status = OCCUPIED;

        nextCar->pPort->tin = getCurrentDate();
        nextCar->inqueue = 0;
        dequeueCar(station->carQueue);

        printf("Car %s moved from queue to charging at port #%d (%s).\n",nextCar->nLicense,port-
>num,portTypeToStr(nextCar->portType));
    }else {
        printf("No available port for next car in queue.\n");
    }
}else {
    printf("No car in queue for port type %s.\n", portTypeToStr(car->portType));
}
}

//Action 5
void dispAllSt(StationBST* stationTree) {
    //printing station in order with summary details.
    inorderPrintStations(stationTree);
}

//Action 6
void dispCarsAtSt(StationBST* stationTree) {
    Station* station = getStationByIdOrName(stationTree);
    if (!station) return;

    Date now = getCurrentDate();
    Port* temp = station->portsList->head;

    printf("Cars at station \"%s\": \n",station->name);
    printf("-----\n");
    printf("InCharge Cars:\n");
    printf("-----\n");
    while (temp) {
        if (temp->status == OCCUPIED && temp->p2car) {
            //check if theirs a car connected to the port and check if the status matches.
            Car* car = temp->p2car;
            int minutes = DateToMinutes(now) - DateToMinutes(temp->tin);
            if (minutes < 0) minutes = 0;

            printf("License: %s, ", car->nLicense);
            printf("Type: %s\n",portTypeToStr(car->portType));
            printf("Charging Time: %d minutes\n", minutes);
        }
        temp = temp->next;
    }
}

```

```

    }
    temp = temp->next;
}
printf("-----\n");
printQueueCars(station->carQueue);
printf("\n");
}

//Action 7
void reportStStat(StationBST* stationTree) {
    Station* station = getStationByIdOrName(stationTree);
    if (!station) return;

    int totalPorts = station->nPorts;
    int outOfOrderPorts = 0;
    int occupiedPorts = 0;

    Port* temp = station->portsList->head;
    while (temp) {
        if (temp->status == OCCUPIED)
            occupiedPorts++;
        if (temp->status == OUT_OF_ORDER)
            outOfOrderPorts++;
        temp = temp->next;
    }

    double utilizationRate = 0.0, DefectiveRatio = 0.0, loadRatio = 0.0;

    //calculates the ports that in use and the faulty ones in precentages.
    if (totalPorts > 0) {
        utilizationRate = 100.0 * occupiedPorts / totalPorts;

        DefectiveRatio = 100.0 * outOfOrderPorts / totalPorts;
    }
    printf("\n--- Station Statistics Report for \"%s\" ---\n", station->name);
    printf("Utilization Rate: %.2f%%\n", utilizationRate);
    printf("Faulty Ports Ratio: %.2f%%\n", DefectiveRatio);

    if (station->nCars > 0) {
        loadRatio = (double)occupiedPorts / station->nCars;
    }
    //classified the load levels.
    printf("Load Level: ");
    if (station->nCars == 0)
        printf("No Load\n");
    else if (loadRatio > 1.0)
        printf("Very Heavy Load\n");
    else if (loadRatio >= 0.9)
        printf("Heavy Load\n");
    else if (loadRatio > 0.2)
        printf("Balanced Load\n");
    else
        printf("Very Light Load\n");
}

//Action 8

```

```

void dispTopCustomers(const CarBST* CarTree) {
    if (!CarTree || !CarTree->root) return;
    //counts recursively the car nodes in the tree.
    int Num_Of_Cars = countCars(CarTree);
    if (Num_Of_Cars == 0)
        return;

    int index = 0;
    //allocate memory for array pointers that will point at the cars.
    Car** topCars = (Car**)malloc(Num_Of_Cars *(sizeof(Car*)));
    if (topCars == NULL) {
        displayError(1);
        return;
    }

    saveCarsInOrder(CarTree->root, topCars, &index); //saving cars to the array.
    qsort(topCars, Num_Of_Cars, sizeof(Car*), compareByPayed); // sorting by total payments.
    printf("top 5 customers with highest total Payment:\n");
    //printing the top 5 cars with most total payments.
    int top = Num_Of_Cars < 5 ? Num_Of_Cars : 5;
    for (int i = 0; i < top; i++) {
        printf("%d. ", i + 1);
        printCar(topCars[i]);
        printf("\n");
    }
    free(topCars); //freeing the pointers array memory.
}

```

//Action 9

```

void addNewPort(StationBST* stationTree) {
    Station* station = getStationByIdOrName(stationTree);
    if (!station) return;

    int portType;
    printf("Enter port type FAST - 0,MID - 1,SLOW - 2 : ");
    scanf("%d", &portType);
    if (portType != FAST && portType != MID && portType != SLOW) {
        displayError(14);
        return;
    }
    Port* newPort = createPort(station->nPorts + 1, portType);
    if (!newPort) {
        displayError(15);
        return;
    }
    addPortToStation(station, newPort); //adding port to station.

    carNode* curr = station->carQueue->front;
    carNode* prev = NULL;

    //handles the case of connecting cars in line to available ports
    while (curr) {
        if (curr->p2car->portType == newPort->portType) {
            if (prev == NULL) {
                station->carQueue->front = curr->next;
            }
        }
    }
}

```

```

        if (station->carQueue->rear == curr)
            station->carQueue->rear = NULL;
    } else {
        prev->next = curr->next;
        if (station->carQueue->rear == curr)
            station->carQueue->rear = prev;
    }

    Car* matchingCar = curr->p2car;
    assignCar2port(matchingCar, newPort);

    free(curr);
    printf("Car %s from queue now charging at port %d (%s).\n",
           matchingCar->nLicense, newPort->num, portTypeToStr(newPort->portType));
    return;
}
prev = curr;
curr = curr->next;
}
printf("No car in queue requires a %s port right now.\n", portTypeToStr(newPort->portType));
}

//Action 10
void releasePorts(StationBST* stationTree) {
    Station* station = getStationByIdOrName(stationTree);
    if (!station) return;
    Date now = getCurrentDate();
    Port* port = station->portsList->head;
    int disconnected = 0;
    int connected = 0;

    while (port) {
        if (port->status == OCCUPIED && port->p2car) {
            int minutes = DateToMinutes(now) - DateToMinutes(port->tin);
            if (minutes >= 600) {
                Car* car = port->p2car;
                printf("Car \"%s\" disconnected. charging time: %d minutes)\n", car->nLicense, minutes);

                //disconnecting car from port
                car->pPort = NULL;
                port->p2car = NULL;
                port->tin = createDate(0, 0, 0, 0, 0);
                port->status = FREE;
                disconnected++;
                //trying to move the line
                Car* nextCar = dequeueCar(station->carQueue);
                if (nextCar) {
                    port->p2car = nextCar;
                    port->tin = now;
                    port->status = OCCUPIED;
                    nextCar->pPort = port;
                    nextCar->inqueue = 0;
                    connected++;
                }
            }
        }
        port = port->next;
    }
}

```

```

    }
    port = port->next;
}
printf("\n");
printf("Connected and disconnected cars in \"%s\":\n",station->name);
printf("%d disconnected cars that is been charge more than 10 hours.\n",disconnected);
printf("%d connected cars from queue.\n",connected);
}

```

//Action 11

```

void remOutOrderPort(StationBST* stationTree) {
    Station* station = getStationByIdOrName(stationTree);
    if (!station) return;

    int stId = station->id;
    Port* port = station->portsList->head;
    int port_num = 0;
    printf("Out of order ports at station #%d:\n", stId);

    int out_of_order = 0;
    //counts and prints out of order ports
    while (port) {
        if (port->status == OUT_OF_ORDER) {
            out_of_order++;
            printf("port #%d, type: %s\n", port->num, portTypeToStr(port->portType));
        }
        port = port->next;
    }
    if (out_of_order == 0) {
        displayError(18);
        return;
    }

    //creating dynamic array for invalid ports numbers.
    int index = 0;
    int* validPorts = (int*)malloc(sizeof(int) * out_of_order);
    if (validPorts == NULL) {
        displayError(1);
        return;
    }
    //adding the out of order ports numbers.
    port = station->portsList->head;
    while (port) {
        if (port->status == OUT_OF_ORDER) {
            validPorts[index++] = port->num;
        }
        port = port->next;
    }
    //loop for deleting the port.
    while (true) {
        printf("\nWhich port would you like to delete?\n");
        printf("(Enter port number, or -1 to cancel): ");
        scanf("%d", &port_num);

        if (port_num == -1) {

```

```

        printf("Operation cancelled. Returning to menu.\n");
        break;
    }

    //verify the selection
    int isValid = 0;
    for (int i = 0; i < out_of_order; i++) {
        if (validPorts[i] == port_num) {
            isValid = 1;
            break;
        }
    }

    //if the input is invalid asks for another chance.
    if (!isValid) {
        printf("Port # %d is not listed as out-of-order.\n", port_num);
        printf("Would you like to try again? (1 = YES / 0 = NO): ");
        int retry;
        scanf("%d", &retry);
        if (retry != 1) {
            printf("Returning to menu.\n");
            break;
        }
        continue;
    }

    if (removePortFromStation(station, port_num)) {
        printf("Port # %d was successfully removed\n", port_num);
        break;
    } else {
        displayError(17);
        printf("Would you like to try again? (1 = YES / 0 = NO): ");
        int retry;
        scanf("%d", &retry);
        if (retry != 1) {
            printf("Returning to menu.\n");
            break;
        }
    }
}

//free dynamic array memory.
free(validPorts);
}

//Action 12
void remCustomer(CarBST* CarTree) {
    if (!CarTree){
        displayError(7);
        return;
    }
    char license[LICENSE_SIZE];
    printf("Enter license(8 characters) or 'exit' to cancel: ");
    scanf(" %8s", license);

    if (strcmp(license, "exit") == 0) {

```

```

        printf("returning to menu\n\n");
        return;
    }
    //saving the amount of cars before removing
    int before = countCarNodes(CarTree->root);
    //trying to remove the car based on license.
    CarTree->root = removeCarNode(CarTree->root, license);
    //check if car removed from the tree.
    int after = countCarNodes(CarTree->root);

    //if there's less cars the removal worked.
    if (after < before) {
        CarTree->totalCars--;
        printf("Car %s was successfully removed\n", license);
    }
}

//Action 13
void closeSt(StationBST *stationTree) {
    Station* station = getStationByIdOrName(stationTree);
    if (!station) return;

    int stId = station->id;
    //check if the cars in queue or in charge process.
    if (station->nCars > 0 || countCarsChargingInStation(station) > 0) {
        displayError(4);
        return;
    }
    //saving the name of the station before removal.
    char closedName[MAX_ST_NAME];
    strcpy(closedName, station->name);

    //removing the station from tree.
    stationTree->root = removeStationNode(stationTree->root, stId);
    printf("Station \"%s\" (ID %d) was successfully closed.\n", closedName, stId);
}

//Action 0
void exitSys(StationBST* stationTree, CarBST* carTree) {
    if (!stationTree || !carTree) {
        displayError(7);
        return;
    }
    //update files and free the memory in the needed order.
    updateFiles(stationTree, carTree);
    freeStationBst(stationTree);
    freeCarsTree(carTree);

    printf("System closed successfully.\n");
}

```

port.h

```

#ifndef PORT_H
#define PORT_H

```



```

#include "date.h"
#include "types.h"
#include <stdbool.h>

typedef struct Car Car;

typedef struct Port {
    int num; //num of port
    portType portType; // charging type(FAST,MID,SLOW)
    portStatus status; // Port status(OCCUPIED, FREE, OUT OF ORDER)
    struct Car* p2car; // Pointer to car
    Date tin; // time of charging
    struct Port* next; // Pointer to next port
}Port;

//Portlist one way linked list
typedef struct {
    Port* head; // pointer to the head of the list
    int count; // totalPorts in the linked lists
}PortList;

/**
 * create and initializes new port
 * @param num of the port
 * @param type of port(FAST,MID,SLOW)
 * @return Pointer to the new port
 */
Port* createPort(int num, portType type);

/**
 * add a new port to the linked list of ports.
 * the new port is appended to the end of the list
 * @param list Pointer to PortList(linked list head)
 * @param newPort Pointer to the new port to add
 */
void addPort(PortList* list, Port* newPort);

/**
 *Searches for available port in the list with the given charging type
 *the port must be free and not out of order
 * @param list Pointer to portlist
 * @param portType desired charging type
 * @return Pointer to an available port or NULL if none found
 */
Port* findAvailablePort(const PortList* list, int portType);

/**
 * print the details of all the ports in the list
 * @param list Pointer to the Portlist
 */
void printPortsList(const PortList* list);

/**

```

```

* removes a port in the list by number
* @param list Pointer to portlist
* @param numToRemove Number of the port to remove
*/
bool removePort(PortList *list, int numToRemove);

/**
* Finds a port in the list by its number
* @param list Pointer to portlist
* @param portNum Pointer to the port to find
* @return Pointer to the port if found, or NULL otherwise
*
* This function removing a port from certain portlist by its number.
*/
Port* findPortByNumber(PortList* list, int portNum);

/**
* Assigns a car to a port and links the port to the car.
* updates all relevant fields:port status, tin(start time), and pointers
* @param car Pointer to the car to assign
* @param port Pointer to the available port to connect the car
*/
bool assignCar2port(Car* car, Port* port);

/**
* Check whether a specific port exists in the list.
* @param list Pointer to portlist
* @param isintheList Pointer to the port to check
* @return 1 if the port exists in the list, 0 otherwise
*/
int isPortInList(const PortList* list, const Port* isintheList);

/**
* free s the entire list
* @param list
*/
void freePortsList(PortList* list);

#endif //PORT_H

port.c
#include "port.h"
#include "car.h"
#include "date.h"
#include "types.h"
#include <stdio.h>
#include <stdlib.h>

#include "Station.h"

Port* createPort(int num, portType type) {
    Port * port = (Port*)malloc(sizeof(Port)); // allocate memory for port.

```

```

    if (port == NULL) {
        return NULL;
    }
    //initializes with the given variables
    port->num = num;
    port->portType = type;
    port->status = FREE;
    port->p2car = NULL;
    port->tin = createDate(0,0,0,0,0);
    port->next = NULL;
return port; //returning the port that we just created.
}

void addPort(PortList* list, Port* newPort) {
    //if the list is empty, we put the new port in the head of the list.
    if (list -> head == NULL) {
        list -> head = newPort;
        //going over th linked list.
    } else {
        Port* temp = list-> head; // temporary pointer to head of list.
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newPort; //adding new port the linked list if temp->next points to null
    }
    list->count++; //updating the amount of ports in portlist.
}

Port* findAvailablePort(const PortList* list, int portType) {
    Port* temp = list->head; //temporary pointer to head of list.
    //running over the linked list.
    while (temp != NULL) {
        //in case the port type match and the status is free it return the available port.
        if (temp->portType == portType && temp->status == FREE) {
            return temp;
        }
        temp = temp->next;
    }
    return NULL; //return NULL in Case port isn't available.
}

void printPortsList(const PortList* list) {
    Port* temp = list -> head;
    printf("Ports:\n");
    while (temp != NULL) {
        printf(" port #%d ", temp->num); //prints port number.
        //prints type and status of the port.
        printf("Type: %s, Status: %s\n", portTypeToStr(temp->portType), portStatusToStr(temp->status));

        temp = temp->next;
    }
}

bool removePort(PortList *list, int numToRemove) {
    if (list == NULL || list->head == NULL) {
        return false; //no option to remove from empty list
    }
}

```

```

}
Port* temp = list->head; //temporary pointer to the current port in the list.
Port* prev = NULL; // pointer to the previous port.
//checking if the first port is the one that needs to be removed
if (temp->num == numToRemove) {
    list->head = temp->next; //update the head of the list.
    free(temp); // freeing the deleted port.
    list->count--; //update the amount of ports in the portlist.
    return true; //removal succeeded
}
// Trying to find the port to remove
while (temp != NULL && temp->num != numToRemove) {
    prev = temp;
    temp = temp->next;
}
// If port was found and we have a valid prev node
// free and remove the node
if (temp != NULL && prev != NULL ) {
    prev->next = temp->next;
    free(temp);
    list->count--;
    return true;
}
return false;
}

```

```

Port* findPortByNumber(PortList* list, int portNum) {
    Port* temp = list -> head;
    //running over the linked list
    while (temp != NULL) {
        if (temp->num == portNum) {
            return temp; // Found the port
        }
        temp = temp->next;
    }
    return NULL; // Port not found
}

```

```

bool assignCar2port(Car* car, Port* port) {
    // validate input parameters
    if (!car || !port) return false;
    // linking car to the port
    car->pPort = port;
    car->inqueue = 0;
    // linking port to car
    port->p2car = car;
    port->status = OCCUPIED;
    port->tin = getCurrentDate();
    return true;
}

```

```

int isPortInList(const PortList* list, const Port* isinthelist) {
    Port* temp = list -> head;
    //running over the linked list comparing port pointers
    while (temp!=NULL) {

```

```

        if (temp == isinthelist) return 1; // Found the port
        temp = temp->next;
    }
    return 0; // port not found
}

```

```

void freePortsList(PortList* list) {
    // validate list pointer
    if (!list) return;
    Port* temp = list->head;
    //running over the linked list and free each port
    while (temp) {
        Port* toFree = temp;
        temp = temp->next;
        // clean up car assignment if port has a car
        if (toFree->p2car) {
            toFree->p2car->pPort = NULL;
            toFree->p2car->inqueue = 0;
            toFree->p2car = NULL;
        }
        free(toFree);
    }
    // reset list to empty state
    list->head = NULL;
    list->count = 0;
}

```

station.h

```

#ifndef STATION_H
#define STATION_H

#include "car.h"
#include "port.h"
#include "carQueue.h"

```

```

typedef struct {
    double x;
    double y;
}Coord;

```

```

typedef struct Station{
    int id;
    char* name;
    int nPorts;
    Coord coord;
    PortList* portsList;
    int nCars;
    qCar* carQueue;
    struct Station* right;
    struct Station* left;
}Station;

```

```

typedef struct {
    Station* root;
    int count;
}

```

```

}StationBST;

/**
 * creates a new station
 * @param id Station id
 * @param name Station name
 * @param coord station coordinates
 * @return Pointer to the new station, NULL if failed
 */
Station* createStation(int id, const char* name, Coord coord);

/**
 * Initializes a StationBST
 * @param tree Pointer to the StationBst
 */
void initStationBst(StationBST* tree);

/**
 * Frees memory allocated for a single station
 */
void freeStation(Station* s);

/**
 * free the station BST.
 */
void freeStationBst(StationBST* tree);

/**
 * Recursively frees station BST nodes
 */
void freeStationBstRecursive(Station* root);

/**
 * Remove a station node from BST by id only if its empty
 * @param station Root of a subtree
 * @param id ID of the station to remove
 * @return Updated root after removal
 */
Station* removeStationNode(Station* station, int id);

/**
 * Inserts a station into the BST
 * @param tree Pointer to the BST
 * @param newSt Pointer to the station to insert
 */
void insertStation(StationBST* tree, Station* newSt);

/**
 * @brief Recursively inserts station into BST
 * @param root Current root node
 * @param newSt Station to insert
 * @return Updated root after insertion
 */
Station* insertStationRecursive(Station* root, Station* newSt);

```

```

/**
 * Prints all stations in BST in order (by ID)
 * @param tree Pointer to BST
 */
void inorderPrintStations(const StationBST* tree);

/**
 * Recursively prints stations in order
 * @param root Current node
 */
void inorderPrintStationsRecursive(Station* root);

/**
 * Prints information about a station
 * @param station Pointer to station to print
 */
void printStation(Station* station);

/**
 * Counts total number of stations in BST
 * @param root Root of BST
 * @return Total number of stations
 */
int countStations(Station* root);

// Port management

/**
 * Adds a port to a station
 * @param s Pointer to station
 * @param p Pointer to port to add
 */
void addPortToStation(Station* s, Port* p);

/**
 * Finds an available port of specified type in station
 * @param s Pointer to station
 * @param type Type of port needed
 * @return Pointer to available port, NULL if none available
 */
Port* findAvailablePortInStation(const Station* s, portType type);

/**
 * Removes a port from station by port number
 * @param s Pointer to station
 * @param portNum Port number to remove
 */
bool removePortFromStation(Station* s, int portNum);

//Car management
/**
 * Counts number of cars currently charging in station
 * @param station Pointer to station
 * @return Number of cars charging

```

```

*/
int countCarsChargingInStation(Station* station);

// Search functions
/**
 * Finds station ID that contains a specific port
 * @param tree Pointer to BST
 * @param port Pointer to port to search for
 * @return Station ID containing the port, 0 if not found
 */
int findStationIdByPort(const StationBST* tree, const Port* port);
/**
 * Recursively searches for station containing a port
 * @param root Current node
 * @param port Port to search for
 * @return Station ID containing the port, 0 if not found
 */
int findStationIdByPortRecursive(const Station* root, const Port* port);
/**
 * Searches for station by ID
 * @param tree Pointer to BST
 * @param id Station ID to search for
 * @return Pointer to the station, NULL if not found
 */
Station* searchStationById(const StationBST* tree, int id);
/**
 * Searches for station by name
 * @param tree Pointer to BST
 * @param name Station ID to search for
 * @return Pointer to the station, NULL if not found
 */
Station* searchStationByName(const StationBST* tree, const char* name);
/**
 * Recursively searches for station by name
 * @param root Current node
 * @param name Name to search for
 * @return Pointer to found station, NULL if not found
 */
Station* searchStationByNameRecursive(const Station* root, const char* name);
/**
 * Calculates distance between two coordinates
 * @param a First coordinate
 * @param b Second coordinate
 * @return Distance between the coordinates
 */
double calculateDistance(Coord a, Coord b);
/**
 * Recursively finds closest station by distance
 * @param root Current node
 * @param location Target location
 * @return Pointer to the closest station
 */
Station* searchStationByDistanceRecursive(const Station* root, Coord location);
/**
 * Finds closest station

```



```

* @param tree Pointer to BST
* @param targetCoord Target coordinates
* @return Pointer to the closest station, NULL if tree is empty
*/
Station* searchStationByDistance(const StationBST* tree, Coord targetCoord);
/**
* Finds station where a specific car is waiting in queue
* @param root Root of BST to search
* @param car Pointer to car to find
* @return Pointer to the station with the car, NULL if not found
*/
Station* findStationByCarInQueue(Station* root, Car* car);

```

```

#endif //STATION_H

```

station.c

```

#define _CRT_SECURE_NO_WARNINGS
#include "Station.h"
#include "port.h"
#include "helper_functions.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

Station* createStation(int id, const char* name, Coord coord) {
    // allocate memory for the new station.
    Station* newStation = (Station*)malloc(sizeof(Station));
    if (newStation == NULL) {
        displayError(1);
        return NULL;
    }
    // Set station ID
    newStation->id = id;
    // Allocate and copy station name
    newStation->name = (char*)malloc(strlen(name) + 1);
    if (newStation->name == NULL) {
        displayError(1);
        free(newStation);
        return NULL;
    }
    strcpy(newStation->name, name);

    newStation->coord = coord;
    newStation->nPorts = 0;
    // Initialize ports list
    newStation->portsList = (PortList*)malloc(sizeof(PortList));
    if (newStation->portsList == NULL) {
        displayError(1);
        free(newStation->name);
        free(newStation);
        return NULL;
    }
    newStation->portsList->head = NULL;
    newStation->portsList->count = 0;
    // Initialize car queue

```

```

newStation->carQueue = (qCar*)malloc(sizeof(qCar));
if (newStation->carQueue == NULL) {
    displayError(1);
    free(newStation->name);
    free(newStation);
    return NULL;
}
initQueue(newStation->carQueue);
newStation->nCars = 0;
// Initialize BST pointers
newStation->left = NULL;
newStation->right = NULL;

return newStation;
}

void initStationBst(StationBST * tree) {
    if (tree) {
        tree->root = NULL;
        tree->count = 0;
    }
}

void freeStation(Station* s) {
    if (s == NULL)
        return;
    //freeing name of the station
    if (s->name != NULL)
        free(s->name);
    //freeing port list
    freePortsList(s->portsList);
    //freeing cars in queue.
    carNode* curr = s->carQueue->front;
    while (curr!= NULL) {
        carNode* toFree = curr;
        curr = curr->next;
        free(toFree);
    }
    // Free the car queue structure
    free(s->carQueue);
    //free the station
    free(s);
}

void freeStationBst(StationBST* tree) {
    if (tree == NULL) return;
    freeStationBstRecursive(tree->root);
    tree->root = NULL;
    tree->count = 0;
}

void freeStationBstRecursive(Station* root) {
    if (root == NULL) return;
    freeStationBstRecursive(root->left);
    freeStationBstRecursive(root->right);
    freeStation(root);
}

```

```

Station* removeStationNode(Station* station, int id) {
    if (!station) return NULL;
    // Search for the node to remove
    if (id < station->id) {
        station->left = removeStationNode(station->left, id);
    }
    else if (id > station->id) {
        station->right = removeStationNode(station->right, id);
    }
    else {
        // Check if the station has cars in it - cannot remove if it does
        if (station->nCars > 0 || countCarsChargingInStation(station) > 0) {
            displayError(4);
            return station;
        }
        // Node with only right child or no child
        if (!station->left) {
            Station* temp = station->right;
            freeStation(station);
            return temp;
        }
        // Node with only left child
        else if (!station->right) {
            Station* temp = station->left;
            freeStation(station);
            return temp;
        }
        // Node with two children find successor
        Station* succParent = station;
        Station* succ = station->right;
        while (succ->left) {
            succParent = succ;
            succ = succ->left;
        }
        // Remove successor from its current position
        if (succParent != station)
            succParent->left = succ->right;
        else
            succParent->right = succ->right;

        // Replace station with successor
        succ->left = station->left;
        succ->right = station->right;

        freeStation(station);
        return succ;
    }
    return station;
}

void insertStation(StationBST* tree, Station* newSt) {
    if (!tree || !newSt) return;
    tree->root = insertStationRecursive(tree->root, newSt);
    tree->count++;
}

```

```

Station* insertStationRecursive(Station* root, Station* newSt) {
    // Validate the new station pointer
    if (newSt == NULL) {
        fprintf(stderr, "invalid station pointer\n");
        return root;
    }
    // If we reached an empty spot in the tree insert there
    if (root == NULL) {
        return newSt;
    }
    // Compare station IDs to decide where to insert
    if (newSt->id < root->id) {
        root->left = insertStationRecursive(root->left, newSt);
    } else if (newSt->id > root->id) {
        root->right = insertStationRecursive(root->right, newSt);
    } else {
        // ID already exist skip
        printf("id already in the system\n");
    }
    return root; // Return the root after insertion
}

Station* searchStationById(const StationBST* tree, int id) {
    // Validate tree and root existence
    if (tree == NULL || tree->root == NULL)
        return NULL;

    Station *temp = tree->root;
    // Iterative search through the BST
    while (temp != NULL) {
        if (id == temp->id) {
            return temp; // found the station
        } else if (id < temp->id) {
            temp = temp->left;
        } else {
            temp = temp->right;
        }
    }
    return NULL; // Station not found
}

Station* searchStationByName(const StationBST* tree, const char* name) {
    // Validate input parameters
    if (tree == NULL || tree->root == NULL || name == NULL)
        return NULL;
    return searchStationByNameRecursive(tree->root, name);
}

Station* searchStationByNameRecursive(const Station* root, const char* name) {
    // reached null node or invalid name
    if (!root || !name) return NULL;
    // Check if current station matches the name
    if (strcmp(root->name, name) == 0) return (Station*)root;
    // Search in left subtree first
    Station* left = searchStationByNameRecursive(root->left, name);
    if (left) return left;
}

```

```

    // search in right if not found in left
    return searchStationByNameRecursive(root->right, name);
}

double calculateDistance(Coord a, Coord b) {
    double xDistance = a.x - b.x;
    double yDistance = a.y - b.y;
    return sqrt(xDistance * xDistance + yDistance * yDistance);
}

Station* searchStationByDistance(const StationBST* tree, Coord targetCoord) {
    // Validate tree structure
    if (tree == NULL || tree->root == NULL) {
        return NULL;
    }
    return searchStationByDistanceRecursive(tree->root, targetCoord);
}

Station* searchStationByDistanceRecursive(const Station* root, Coord location) {
    if (!root) return NULL;
    // Initialize current station as closest and calculate its distance
    Station * closestStation = (Station*)root;
    double minDistance = calculateDistance(root->coord, location);
    // Check left for closer stations
    Station* left = searchStationByDistanceRecursive(root->left, location);
    if (left) {
        double leftDistance = calculateDistance(left->coord, location);
        if (leftDistance < minDistance) {
            minDistance = leftDistance;
            closestStation = left;
        }
        // Check right for closer stations
    }
    Station* right = searchStationByDistanceRecursive(root->right, location);
    if (right) {
        double rightDistance = calculateDistance(right->coord, location);
        if (rightDistance < minDistance) {
            minDistance = rightDistance;
            closestStation = right;
        }
    }
    return closestStation;
}

void inorderPrintStations(const StationBST* tree) {
    if (tree == NULL) return;
    inorderPrintStationsRecursive(tree->root);
}

void inorderPrintStationsRecursive(Station* root) {
    if (root == NULL) return;
    inorderPrintStationsRecursive(root->left);
    printStation(root);
    inorderPrintStationsRecursive(root->right);
}

```

```

void printStation(Station* station) {
    // Validate station pointer
    if (station == NULL) {
        printf("station is NULL\n");
        return;
    }
    int waiting = station->nCars; // Cars waiting in queue
    int inCharge = countCarsChargingInStation(station); // Cars currently charging
    printf("=====\n");
    printf("Station name: %s \n", station->name);
    printf("Station id: %d \n", station->id);
    printf("Station Coordinates: %0.2f, %0.2f \n", station->coord.x, station->coord.y);
    printPortsList(station->portsList); // Print ports in station
    printf("Amount of cars in queue: %d\n", waiting);
    printf("Amount of cars in charge %d\n", inCharge);
    printf("=====\n\n");
}

```

```

int countStations(Station* root) {
    if (!root) return 0;
    // Count current node + left subtree + right subtree
    return 1 + countStations(root->left) + countStations(root->right);
}

```

```

void addPortToStation(Station* s, Port* p) {
    // Validate input parameters
    if (!s || !p) return;
    // Add port to the station's port list
    addPort(s->portsList, p);
    s->nPorts++;
}

```

```

Port* findAvailablePortInStation(const Station* s, portType type) {
    // Validate station pointer
    if (s == NULL)
        return NULL;
    Port* temp = s->portsList->head;
    while (temp != NULL) {
        // Check if port is free and matches requested type
        if (temp->status == FREE && temp->portType == type) {
            return temp; // Found matched port
        }
        temp = temp->next; // Move to the next port
    }
    return NULL;
}

```

```

bool removePortFromStation(Station* s, int portNum) {
    // Validate station pointer
    if (s == NULL)
        return false;
    // remove port from the station's port list
}

```

```

    if (removePort(s->portsList, portNum)) {
        s->nPorts--;
        return true;
    }
    return false;
}

int findStationIdByPort(const StationBST* tree, const Port* port) {
    // Validate input parameters
    if (!tree || !port) return 0;
    return findStationIdByPortRecursive(tree->root, port);
}

int findStationIdByPortRecursive(const Station* root, const Port* port) {
    if (!root || !port) return 0;
    // Check if current station contains the port
    if (isPortInList(root->portsList, port))
        return root->id;
    // Search in left
    int left = findStationIdByPortRecursive(root->left, port);
    if (left) return left;
    // Search in right if not found in left
    int right = findStationIdByPortRecursive(root->right, port);
    return right;
}

Station* findStationByCarInQueue(Station* root, Car* car) {
    // Validate input parameters
    if (!root || !car) return NULL;
    // Search in left first
    Station* left = findStationByCarInQueue(root->left, car);
    if (left) return left;
    // Check current station's car queue
    carNode* temp = root->carQueue->front;
    while (temp) {
        if (temp->p2car == car)
            return root; // Found car in this station's queue
        temp = temp->next;
    }
    // Search in right if not found
    return findStationByCarInQueue(root->right, car);
}

int countCarsChargingInStation(Station* station) {
    // Validate station pointer
    if (!station) return 0;
    int InCharge = 0;
    Port* temp = station->portsList->head;
    while (temp) {
        // Count ports that are currently occupied
        if (temp->status == OCCUPIED) {
            InCharge++;
        }
        temp = temp->next;
    }
}

```

```

    }
    return InCharge;
}
types.h
#ifndef TYPES_H
#define TYPES_H

typedef enum portType {
    FAST,
    MID,
    SLOW
}portType;

typedef enum portStatus{
    OCCUPIED = 1,
    FREE = 2,
    OUT_OF_ORDER =3
}portStatus;

const char* portTypeToStr(enum portType type);
const char* portStatusToStr(enum portStatus status);

portType portTypeFromStr(const char* str);

#endif //TYPES_H

```

```

types.c
#include <stdio.h>
#include <stdlib.h>
#include "types.h"
#include <string.h>

const char* portTypeToStr(enum portType type) {
    switch (type) {
        case FAST:
            return "FAST";

        case MID:
            return "MID";

        case SLOW:
            return "SLOW";
        default:
            return "Unknown Type";
    }
}

portType portTypeFromStr(const char* str) {
    if (strcmp(str, "FAST") == 0) return FAST;
    if (strcmp(str, "MID") == 0) return MID;
    if (strcmp(str, "SLOW") == 0) return SLOW;
    return -1;
}

```



```

const char* portStatusToStr(enum portStatus status) {
    switch (status) {
        case OCCUPIED:
            return "OCCUPIED";

        case FREE:
            return "FREE";

        case OUT_OF_ORDER:
            return "OUT_OF_ORDER";
        default:
            return "Unknown Type";
    }
}

```

main.c

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include "car.h"
#include "Station.h"
#include "files.h"
#include "menu.h"
#include "menu_functions.h"

#define EXIT 0

int main() {
    int User_Choice;
    int result = 0;
    StationBST stationTree;
    initStationBst(&stationTree); // intializes station bst
    CarBST* carTree = createCarBST(); //creating car BST
    loadFiles(carTree, &stationTree);
    do {
        Menu();
        result = scanf("%d", &User_Choice);
        if (result != 1) {
            printf("Invalid Choice, Please enter a valid number.\n\n");
            while (getchar() != '\n')continue;
            continue;
        }
        clear();
        switch (User_Choice) {
            case 1: {
                locNearSt(&stationTree);
                break;
            }
            case 2: {
                chargeCar(&stationTree, carTree);
                updateFiles(&stationTree, carTree);
                break;
            }
        }
    }
}

```

```
case 3: {
    checkCarStatus(&stationTree,carTree);
    break;
}
case 4: {
    stopCharge(&stationTree,carTree);
    updateFiles(&stationTree,carTree);
    break;
}
case 5: {
    dispAllSt(&stationTree);
    break;
}
case 6: {
    dispCarsAtSt(&stationTree);
    break;
}
case 7: {
    reportStStat(&stationTree);
    break;
}
case 8: {
    dispTopCustomers(carTree);
    break;
}
case 9: {
    addNewPort(&stationTree);
    updateFiles(&stationTree,carTree);
    break;
}
case 10: {
    releasePorts(&stationTree);
    updateFiles(&stationTree,carTree);
    break;
}
case 11: {
    remOutOrderPort(&stationTree);
    updateFiles(&stationTree,carTree);
    break;
}
case 12: {
    remCustomer(carTree);
    updateFiles(&stationTree,carTree);
    break;
}
case 13: {
    closeSt(&stationTree);
    updateFiles(&stationTree,carTree);
    break;
}
case 0: {
    exitSys(&stationTree,carTree);
}
default: {
    break;
}
```

```

    }
}
    printf("\n");
} while (User_Choice != EXIT);
return 0;
}

```

דוח בדיקות:

locNearSt

```

Enter coordinates(x,y) separated by commas: 30 30
Invalid input, Please enter two numbers seperated by commas
Enter coordinates(x,y) separated by commas: nir
Invalid input, Please enter two numbers seperated by commas
Enter coordinates(x,y) separated by commas: nir,nir
Invalid input, Please enter two numbers seperated by commas
Enter coordinates(x,y) separated by commas: 30#,30
Invalid input, Please enter two numbers seperated by commas
Enter coordinates(x,y) separated by commas: 30
Invalid input, Please enter two numbers seperated by commas
Enter coordinates(x,y) separated by commas: 32.1,34.8

The nearest station: Tel Aviv Center
Distance: 0.02
Total ports: 4
Active ports: 4
Busy ports (with car): 4
Cars waiting in queue: 0
Port types:
    SLOW: 1
    MID : 2
    FAST: 1

```

	ID,StationName,NumOfPorts,CoordX,CoordY
FAST,1,2025,6,30,14,30,12345678,101,1	Tel Aviv Center,4,32.09,34.78,101
MID,1,2025,6,30,14,32,87654321,101,2	Haifa Port,3,32.79,34.99,205
SLOW,1,2025,6,30,14,35,23456789,101,3	Beer Sheva North,5,31.25,34.79,307
MID,1,2025,6,30,14,37,34567890,101,4	Jerusalem East,2,31.77,35.21,412
	Herzliya Marina,6,32.16,34.81,518
	Ashdod South,4,31.80,34.64,623

אם המשתמש לא מכניס בדיוק שני מספרים חוקיים מופרדים בפסיק, הפונקציה מדפיסה הודעת שגיאה, מנקה את הבאפר, ומבקשת שוב קלט.

chargeCar

```

Search station by: ID-1, Name- 2? : 1
Enter station ID or -1 to cancel: 101
Enter license(8 characters) or 'exit' to cancel: 28173903
Car 28173903 not found. Please enter its details:
Enter port type(SLOW/MID/FAST): MID
Car 28173903 added to system.
Error: No available port of required type.
Car 28173903 has entered the queue at station Tel Aviv Center.

```

License,StationID

28173903,101

28173908,412

25045665,518

29587102,518

```
Search station by: ID-1, Name- 2? : d
Invalid input. please enter 1 or 2.
Search station by: ID-1, Name- 2? : 3
Invalid input. please enter 1 or 2.
Search station by: ID-1, Name- 2? : -1
Invalid input. please enter 1 or 2.
Search station by: ID-1, Name- 2? : 1
Enter station ID or -1 to cancel: nir
Invalid station ID. please enter an ID
Enter station ID or -1 to cancel: 1000
Error: Station not found.
```

המשתמש צריך להכניס קודם כל 1 או 2 עד שיתקבל קלט חוקי ולאחר מכן מזהה תחנה תקין, אחרת יוצא מהמערכת.

checkCarStatus

```
Enter license(8 characters) or 'exit' to cancel: 2817390#
Error: Car not found.
```

```
Enter license(8 characters) or 'exit' to cancel: 28173903
Car 28173903 is in queue at station "Tel Aviv Center"
Position in MID charging queue: 1.
```

```
Enter license(8 characters) or 'exit' to cancel: 210
Error: Car not found.
```

```
Enter license(8 characters) or 'exit' to cancel: 2222222
Error: Car not found.
```

אם המשתמש לא מכניס מספר רכב תקין(מספר בעל 8 ספרות) מודפסת הודעת שגיאה.

stopCharge

```
Enter license(8 characters) or 'exit' to cancel: 606085
Error: Car not found.
```

```
Enter license(8 characters) or 'exit' to cancel: 28173903
Error: Car is not currently charging.
```

```
Enter license(8 characters) or 'exit' to cancel: 11223344
Car 11223344 finished charging at station "Jerusalem East", port #1.
Charging time: 50551 minutes. price to pay: 60661.20 ILS.
Car 28173908 moved from queue to charging at port #1 (FAST).
```

License,PortType,TotalPaid,StationID,PortNumber,InQueue
FAST,60752.00,0,0,0,11223344

28173908,FAST,93.45,412,1,0

License ,StationID

28173903,101

25045665,518

29587102,518

המשתמש חייב להכניס אך ורק רכב שכבר נמצא בטעינה מוציא אותו מטעינה ושם במקומו רכב שהיה בתור.

dispAllSt

```
=====
Station name: Herzliya Marina
Station id: 518
Station Coordinates: 32.16, 34.81
Ports:
  port #1 Type: MID, Status: OCCUPIED
  port #2 Type: SLOW, Status: FREE
  port #3 Type: MID, Status: FREE
  port #4 Type: FAST, Status: OCCUPIED
  port #5 Type: MID, Status: FREE
  port #6 Type: SLOW, Status: FREE
Amount of cars in queue: 2
Amount of cars in charge 2
=====

=====
Station name: Beer Sheva North
Station id: 307
Station Coordinates: 31.25, 34.79
Ports:
  port #1 Type: FAST, Status: FREE
  port #2 Type: FAST, Status: FREE
  port #3 Type: SLOW, Status: OUT_OF_ORDER
  port #4 Type: MID, Status: FREE
  port #5 Type: MID, Status: FREE
Amount of cars in queue: 0
Amount of cars in charge 0
=====

=====
Station name: Ashdod South
Station id: 623
Station Coordinates: 31.80, 34.64
Ports:
  port #1 Type: MID, Status: OCCUPIED
  port #2 Type: FAST, Status: FREE
  port #3 Type: SLOW, Status: OUT_OF_ORDER
  port #4 Type: MID, Status: FREE
Amount of cars in queue: 0
Amount of cars in charge 1
=====

=====
Station name: Jerusalem East
Station id: 412
Station Coordinates: 31.77, 35.21
Ports:
  port #1 Type: FAST, Status: OCCUPIED
  port #2 Type: MID, Status: FREE
Amount of cars in queue: 0
Amount of cars in charge 1
=====

=====
Station name: Tel Aviv Center
Station id: 101
Station Coordinates: 32.09, 34.78
Ports:
  port #1 Type: FAST, Status: OCCUPIED
  port #2 Type: MID, Status: OCCUPIED
  port #3 Type: SLOW, Status: OCCUPIED
  port #4 Type: MID, Status: OCCUPIED
Amount of cars in queue: 1
Amount of cars in charge 4
=====

=====
Station name: Haifa Port
Station id: 205
Station Coordinates: 32.79, 34.99
Ports:
  port #1 Type: FAST, Status: FREE
  port #2 Type: MID, Status: FREE
  port #3 Type: SLOW, Status: FREE
Amount of cars in queue: 0
Amount of cars in charge 0
=====
```

dispCarsAtSt

```
Search station by: ID-1, Name- 2? : 3
Invalid input. please enter 1 or 2.
Search station by: ID-1, Name- 2? : d
Invalid input. please enter 1 or 2.
Search station by: ID-1, Name- 2? : 1
Enter station ID or -1 to cancel: 1000
Error: Station not found.
```

```
Search station by: ID-1, Name- 2? : 1
Enter station ID or -1 to cancel: 101
Cars at station "Tel Aviv Center":
-----
InCharge Cars:
-----
License: 12345678 ,Type: FAST
Charging Time: 50328 minutes
License: 87654321 ,Type: MID
Charging Time: 50326 minutes
License: 23456789 ,Type: SLOW
Charging Time: 50323 minutes
License: 34567890 ,Type: MID
Charging Time: 50321 minutes
-----
Cars in queue:
-----
Car 1: License: 28173903 ,Type: MID
-----
```

המתשמש צריך להכניס מזהה או שם תחנה תקין.

reportStStat

```
Search station by: ID-1, Name- 2? : 1
Enter station ID or -1 to cancel: 101

--- Station Statistics Report for "Tel Aviv Center"
Utilization Rate: 100.00%
Faulty Ports Ratio: 0.00%
Load Level: Very Heavy Load
```

המתשמש צריך להכניס מזהה או שם תחנה תקין.

dispTopCustomers

```
top 5 customers with highest total Payment:
1. Car, 11223344
total payed:60752.00

2. Car, 22112748
total payed:147.73

3. Car, 26920954
total payed:142.62

4. Car, 27187646
total payed:133.23

5. Car, 12345678
total payed:125.50
```

addNewPort

```
Search station by: ID-1, Name- 2? : 2
Enter station name or exit to cancel: Tel Aviv
Error: Station not found.
```

```
Search station by: ID-1, Name- 2? : 2
Enter station name or exit to cancel: Tel Aviv Center
Enter port type FAST - 0,MID - 1,SLOW - 2 : MID
Error: Invalid port type.
```

```
Search station by: ID-1, Name- 2? : 2
Enter station name or exit to cancel: Tel Aviv Center
Enter port type FAST - 0,MID - 1,SLOW - 2 : 0
No car in queue requires a FAST port right now.
```

```
Search station by: ID-1, Name- 2? : 2
Enter station name or exit to cancel: Tel Aviv Center
Enter port type FAST - 0,MID - 1,SLOW - 2 : 1
Car 28173903 from queue now charging at port 6 (MID).
```

המשתמש צריך להכניס מזהה או שם תחנה תקין וגם קצב טעינה בפורמט הנכון, בנוסף רק אם יש מכונית בתור של התחנה המבוקשת שמתאים לה קצב הטעינה המבוקש הפונקציה תיצור עמדת טעינה חדשה ותטעין את הרכב המתאים בתור.

releasePorts

```
Search station by: ID-1, Name- 2? : 2
Enter station name or exit to cancel: Tel Aviv Center
Car "12345678" disconnected. charging time: 50363 minutes)
Car "87654321" disconnected. charging time: 50361 minutes)
Car "23456789" disconnected. charging time: 50358 minutes)
Car "34567890" disconnected. charging time: 50356 minutes)
```

```
Connected and disconnected cars in "Tel Aviv Center":
4 disconnected cars that is been charge more than 10 hours.
0 connected cars from queue.
```

```
Search station by: ID-1, Name- 2? : 1
Enter station ID or -1 to cancel: 518
Car "55667788" disconnected. charging time: 50565 minutes)
Car "66778899" disconnected. charging time: 50535 minutes)
```

```
Connected and disconnected cars in "Herzliya Marina":
2 disconnected cars that is been charge more than 10 hours.
2 connected cars from queue.
```

הפונקציה תנתק את הרכבים שנמצאים מעל 10 שעות בתחנה ותחבר את אלו שבתור.

```
Search station by: ID-1, Name- 2? : 1
Enter station ID or -1 to cancel: 518
Cars at station "Herzliya Marina":
```

```
-----
InCharge Cars:
```

```
-----
License: 25045665 ,Type: FAST
Charging Time: 7 minutes
License: 29587102 ,Type: FAST
Charging Time: 7 minutes
-----
```

```
Queue is empty
```


remOutOrderPort

```
=====
Station name: Ashdod South
Station id: 623
Station Coordinates: 31.80, 34.64
Ports:
    port #1 Type: MID, Status: OCCUPIED
    port #2 Type: FAST, Status: FREE
    port #3 Type: SLOW, Status: OUT_OF_ORDER
    port #4 Type: MID, Status: FREE
Amount of cars in queue: 0
Amount of cars in charge 1
=====
```

```
Search station by: ID-1, Name- 2? : 1
Enter station ID or -1 to cancel: 623
Out of order ports at station #623:
port #3, type: SLOW

Which port would you like to delete?
(Enter port number, or -1 to cancel): 3
Port #3 was successfully removed
```

```
=====
Station name: Ashdod South
Station id: 623
Station Coordinates: 31.80, 34.64
Ports:
    port #1 Type: MID, Status: OCCUPIED
    port #2 Type: FAST, Status: FREE
    port #4 Type: MID, Status: FREE
Amount of cars in queue: 0
Amount of cars in charge 1
=====
```

הפונקציה מוחקת את העמדה התקולה המבוקשת.

remCustomer

```
Search station by: ID-1, Name- 2? : 1
Enter station ID or -1 to cancel: 101
Cars at station "Tel Aviv Center":
-----
InCharge Cars:
-----
License: 28173903 ,Type: MID
Charging Time: 33 minutes
-----
Queue is empty
```

```
Enter license(8 characters) or 'exit' to cancel: 28173903
car 28173903 is waiting in line or charging
```

```
Enter license(8 characters) or 'exit' to cancel: 28173903
Car 28173903 finished charging at station "Tel Aviv Center", port #6.
Charging time: 35 minutes. price to pay: 42.00 ILS.
No car in queue for port type MID.
```

```
Enter license(8 characters) or 'exit' to cancel: 28173903
Car 28173903 was successfully removed
```

```
Search station by: ID-1, Name- 2? : 1
Enter station ID or -1 to cancel: 101
Cars at station "Tel Aviv Center":
-----
InCharge Cars:
-----
-----
Queue is empty
```

בהתחלה הרכב היה באמצע טעינה בתחנה ולכן לא היה ניתן למחוק את הרכב, באמצעות פעולה 4 הפסקתי את הטעינה והפעם כן היה ניתן למחוק את הרכב מהתחנה.

closeSt

```
ID,StationName,NumOfPorts,CoordX,CoordY
101,Tel Aviv Center,7,32.09,34.78
205,Haifa Port,3,32.79,34.99
307,Beer Sheva North,5,31.25,34.79
412,Jerusalem East,2,31.77,35.21
518,Herzliya Marina,6,32.16,34.81
623,Ashdod South,3,31.80,34.64
```

```
Search station by: ID-1, Name- 2? : 1
Enter station ID or -1 to cancel: 101
Station "Tel Aviv Center" (ID 101) was successfully closed.
```

```
Search station by: ID-1, Name- 2? : 1
Enter station ID or -1 to cancel: 623
Error: Station cannot be removed (cars charging or in queue).
```

```
ID,StationName,NumOfPorts,CoordX,CoordY
205,Haifa Port,3,32.79,34.99
307,Beer Sheva North,5,31.25,34.79
412,Jerusalem East,2,31.77,35.21
518,Herzliya Marina,6,32.16,34.81
623,Ashdod South,3,31.80,34.64
```

רק אם בתחנה אין רכבים ניתן למחוק אותה.

exitSys

```
System closed successfully.
```

```
Enter coordinates(x,y) separated by commas: 32.1,34.8
```

```
Error: Station not found.
```

```
Search station by: ID-1, Name- 2? : 1
Enter station ID or -1 to cancel: 101
Error: Station not found.
```

אחרי שינוי במיין שהתוכניות לא תסגר בלחיצה על 0, אפשר להמשיך ולהפעיל פעולות
אחרות כמו 1 ו 2 ולבדוק אם באמת היה שחרור זכרון, אפשר לראות שאין אפשרות לגשת
לתחנות ושהשחרור בוצע בהצלחה.

